

RAG Document Assistant

Documentación Técnica Completa

Python · FastAPI · LangChain · ChromaDB · OpenAI

Versión 1.0 · June 2026

Este documento cubre en profundidad cada componente del sistema RAG: arquitectura completa, código fuente comentado línea por línea, conceptos técnicos fundamentales con analogías, decisiones de diseño justificadas y glosario. Escrito para quien quiere entender no solo el qué, sino el cómo y el por qué.

Índice de Contenidos

1. Introducción al sistema RAG

- 1.1 ¿Qué es RAG?
- 1.2 El problema que resuelve
- 1.3 Cómo funciona conceptualmente
- 1.4 RAG vs. alternativas

2. Arquitectura general del proyecto

- 2.1 Diagrama de flujo de datos
- 2.2 Las dos fases del sistema
- 2.3 Componentes y tecnologías

3. Análisis detallado — archivo por archivo

- 3.1 `app/config.py` — Configuración
- 3.2 `app/ingest.py` — Ingesta de documentos
- 3.3 `app/query.py` — Consulta RAG
- 3.4 `app/main.py` — API FastAPI

4. Flujo completo de una request

- 4.1 Fase 1: de PDF a vector (ingesta)
- 4.2 Fase 2: de pregunta a respuesta (consulta)

5. Conceptos clave en profundidad

- 5.1 Embeddings y representación vectorial
- 5.2 Similarity search y cosine similarity
- 5.3 Chunking y overlap
- 5.4 Prompt engineering
- 5.5 LCEL: LangChain Expression Language

6. Dependencias y librerías

7. Decisiones de diseño

8. Glosario de términos técnicos

1. Introducción al sistema RAG

1.1 ¿Qué es RAG?

RAG (Retrieval-Augmented Generation — Generación Aumentada por Recuperación) es una arquitectura de inteligencia artificial que combina dos capacidades: buscar información relevante en una base de datos (retrieval) y generar texto coherente y contextualizado usando un modelo de lenguaje (generation).

Analogía fundamental: Imaginá un estudiante universitario extremadamente inteligente (el LLM). Sabe todo lo que estudió durante su carrera, pero no puede saber el contenido de un libro que nunca leyó. RAG es equivalente a darle ese libro antes del examen: primero lo hojea (retrieval), encuentra los párrafos relevantes, y luego responde la pregunta con esa información fresca (generation).

1.2 El problema que resuelve

Los Modelos de Lenguaje Grande (LLMs) como GPT tienen dos limitaciones estructurales que RAG resuelve directamente:

- **Conocimiento estático:** el modelo solo sabe lo que aprendió durante su entrenamiento (generalmente hasta una fecha de corte). No puede responder preguntas sobre eventos recientes ni sobre información que no estaba disponible públicamente.
- **Sin acceso a documentos privados:** el modelo nunca vio tu documentación interna, tus contratos, tus manuales técnicos, ni tus PDFs corporativos. No puede responder preguntas sobre ellos.
- **Alucinaciones:** cuando el modelo no sabe algo, tiende a inventar respuestas que suenan plausibles pero son incorrectas. RAG lo ancla a fuentes reales.

Ejemplo concreto: Si le preguntás a GPT '¿Cuál es la política de vacaciones de mi empresa?' va a inventar una respuesta genérica. Con RAG, primero buscamos en el reglamento interno de la empresa, encontramos el párrafo exacto, y GPT responde con esa información real.

1.3 Cómo funciona conceptualmente

El sistema opera en dos fases completamente separadas:

FASE 1 — Ingesta (se ejecuta una vez por documento):

- Leer el documento PDF completo
- Dividirlo en fragmentos de texto manejables (chunks)
- Convertir cada fragmento en un vector numérico (embedding)
- Guardar esos vectores en una base de datos vectorial (ChromaDB)

FASE 2 — Consulta (se ejecuta ante cada pregunta del usuario):

- Recibir la pregunta del usuario en lenguaje natural
- Convertir la pregunta en un vector numérico (con el mismo modelo de embeddings)
- Buscar en ChromaDB los fragmentos cuyo vector es más similar al de la pregunta
- Construir un prompt que incluye esos fragmentos como contexto
- Enviar el prompt al LLM (GPT) para que genere la respuesta final
- Retornar la respuesta junto con las fuentes utilizadas

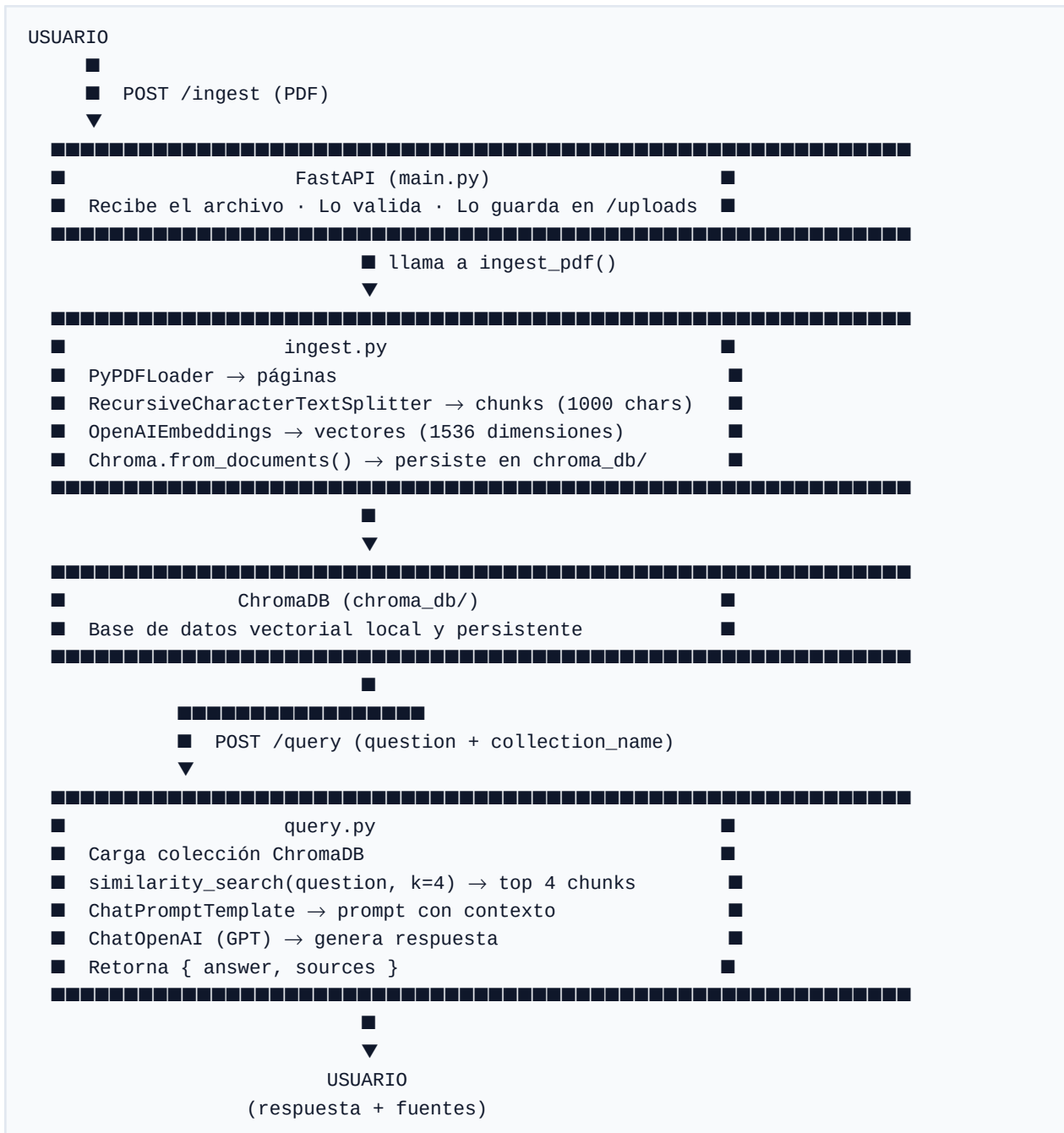
1.4 RAG vs. alternativas

Técnica	Ventajas	Desventajas
RAG	Económico, actualizable, fuentes explícitas, sin reentrenamiento	Depende de la calidad del retrieval
Fine-tuning	El modelo 'incorpora' el conocimiento	Costoso, lento, difícil de actualizar, puede olvidar
Context stuffing	Simple de implementar	Limitado por el context window, muy caro en tokens

2. Arquitectura General del Proyecto

2.1 Diagrama de flujo de datos

El siguiente diagrama muestra el flujo completo de datos del sistema, desde que el usuario sube un PDF hasta que recibe una respuesta:



2.2 Las dos fases del sistema

Es fundamental entender que el sistema opera en dos momentos distintos y separados. La fase de ingesta puede ocurrir horas o días antes de la primera consulta, y solo necesita ejecutarse una vez por documento. La fase de consulta es la que el usuario experimenta en tiempo real.

Analogía de la biblioteca: La ingesta es como el trabajo del bibliotecario que ficha y ordena los libros cuando llegan. La consulta es cuando un estudiante llega, pide información, el bibliotecario sabe exactamente qué páginas buscar y se las entrega. El estudiante (GPT) lee esas páginas y responde la pregunta.

2.3 Componentes y tecnologías

Componente	Tecnología	Rol
API REST	FastAPI	Expone endpoints HTTP al frontend
Carga de PDFs	PyPDFLoader	Lee y extrae texto página por página
Chunking	RecursiveCharacterText Splitter	Divide texto en fragmentos manejables
Embeddings	OpenAI text-embedding-ada-002	Convierte texto a vectores de 1536 dims
Vector DB	ChromaDB	Almacena y busca vectores localmente
LLM	GPT-3.5-turbo / GPT-4	Genera la respuesta final en lenguaje natural
Orquestación	LangChain (LCEL)	Conecta y encadena todos los componentes

3. Análisis Detallado — Archivo por Archivo

3.1 app/config.py — Configuración del Sistema

Este módulo cumple una función aparentemente simple pero crítica: centralizar toda la configuración del sistema en un único lugar. Sin este archivo, cada módulo tendría que importar **python-dotenv** y gestionar sus propias variables, lo que generaría duplicación de código y haría el sistema frágil.

Código completo del módulo:

```
# Carga y expone la configuración de la aplicación desde variables de entorno.
# OpenAI se usa tanto para embeddings como para generación de respuestas.

import os                                # Módulo estándar: acceso a variables de entorno del SO
from dotenv import load_dotenv           # Lee el archivo .env y carga sus pares CLAVE=valor

load_dotenv() # Ejecuta la lectura del .env. Si no existe, no lanza error (modo silencioso)

def _require(name: str) -> str:
    value = os.getenv(name)              # Busca la variable en el entorno del proceso actual
    if value is None:                     # None significa que no fue encontrada (distinto de cadena vacía)
        raise ValueError(
            f"Variable de entorno requerida no encontrada: '{name}'. "
            f"Asegúrate de definirla en tu archivo .env o en el entorno del sistema."
        )
    return value                          # Si existe, retorna su valor como string

OPENAI_API_KEY    = _require("OPENAI_API_KEY")    # Falla inmediatamente si no está definida
CHROMA_PERSIST_DIR = _require("CHROMA_PERSIST_DIR") # Ruta donde ChromaDB guarda los datos
UPLOAD_DIR        = _require("UPLOAD_DIR")        # Ruta donde se guardan los PDFs subidos
OPENAI_MODEL      = os.getenv("OPENAI_MODEL", "gpt-3.5-turbo") # Opcional: default gpt-3.5-turbo
```

Análisis línea por línea:

import os

El módulo **os** de Python provee acceso al sistema operativo. En este caso usamos **os.getenv(name)**, que busca una variable de entorno por nombre. Las variables de entorno son pares clave-valor que el sistema operativo mantiene para cada proceso. Cuando ejecutás uvicorn, hereda las variables de entorno de tu terminal, incluyendo las que **load_dotenv()** inyectó desde **.env**.

from dotenv import load_dotenv

python-dotenv lee el archivo **.env** (si existe) y agrega cada par **CLAVE=valor** al entorno del proceso usando **os.environ**. Esto permite que **os.getenv()** las encuentre normalmente. Sin esta línea, solo funcionaría si el sistema operativo ya tuviera esas variables definidas (como

en producción con variables de servidor).

La función `_require()`

El prefijo `_` indica que es una función privada del módulo (convención de Python). Su propósito es el principio DRY (Don't Repeat Yourself): en lugar de escribir la misma validación cuatro veces, la encapsulamos una vez. Usa **`ValueError`** y no **`KeyError`** porque `ValueError` comunica mejor que el problema es de configuración del usuario, no un error de programación.

¿Por qué `OPENAI_MODEL` usa `os.getenv()` con `default` y no `_require()`?

`OPENAI_MODEL` es opcional porque tiene un valor sensato por defecto: `gpt-3.5-turbo`. El usuario puede no definirla y el sistema funciona igual. En cambio, `OPENAI_API_KEY` no tiene default posible — sin ella, ninguna llamada a OpenAI puede funcionar. Esta distinción entre 'obligatorio' y 'opcional con default' es un patrón de diseño de configuración muy común.

¿Por qué las constantes están a nivel de módulo y no dentro de funciones?

Al estar a nivel de módulo, la validación ocurre en el momento del `import`. Si falta una variable, el servidor falla al arrancar con un mensaje claro, en lugar de fallar silenciosamente en la primera request que llegue. Esto implementa el principio 'fail fast': fallar pronto y con claridad.

3.2 app/ingest.py — Ingesta de Documentos PDF

Este módulo implementa el pipeline completo de ingesta: toma un archivo PDF del disco, lo convierte en fragmentos de texto, genera embeddings para cada fragmento y los almacena en ChromaDB. Es el corazón de la fase de preparación del sistema RAG.

```
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import Chroma
from app.config import OPENAI_API_KEY, CHROMA_PERSIST_DIR

def ingest_pdf(file_path: str) -> dict:
    # PASO 1: extraer el nombre del archivo sin extensión (será el nombre de la colección)
    filename = os.path.splitext(os.path.basename(file_path))[0]

    # PASO 2: cargar el PDF (una página = un Document de LangChain)
    loader = PyPDFLoader(file_path)
    pages = loader.load()          # Lista de objetos Document, uno por página
    total_pages = len(pages)

    # PASO 3: dividir en fragmentos con solapamiento
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=1000,          # Máximo 1000 caracteres por fragmento
        chunk_overlap=200,        # Los últimos 200 chars de un chunk se repiten al inicio del siguiente
    )
    chunks = splitter.split_documents(pages)
    total_chunks = len(chunks)

    # PASO 4: inicializar el modelo de embeddings
    embeddings = OpenAIEmbeddings(openai_api_key=OPENAI_API_KEY)

    # PASO 5: vectorizar y persistir en ChromaDB
    Chroma.from_documents(
        documents=chunks,
        embedding=embeddings,
        collection_name=filename,
        persist_directory=CHROMA_PERSIST_DIR,
    )

    return {
        "filename": filename,
        "total_pages": total_pages,
        "total_chunks": total_chunks,
        "collection_name": filename,
    }
```

PyPDFLoader — Cargador de PDFs

PyPDFLoader usa la librería **pypdf** internamente para leer el archivo PDF. Retorna una lista de objetos **Document** de LangChain, donde cada Document contiene el texto de una página y

un diccionario de metadata con información como número de página, nombre del archivo, autor, fecha de creación, etc. Esta metadata viaja junto al texto durante todo el pipeline y aparece en la respuesta final como 'fuentes'.

RecursiveCharacterTextSplitter — El chunking

Este es el componente más crítico para la calidad del sistema. Su función es dividir textos largos en fragmentos más pequeños que puedan ser vectorizados y recuperados individualmente.

¿Por qué `chunk_size=1000`?

1000 caracteres ≈ 150-200 palabras ≈ 1-2 párrafos de texto normal.

Este tamaño es el resultado de balancear tres factores:

1. **Contexto suficiente:** un fragmento de 1000 chars contiene suficiente información para que el LLM pueda responder preguntas sobre él.
2. **Precisión del retrieval:** fragmentos más pequeños son más específicos y el similarity search los encuentra mejor. Fragmentos de 5000 chars contienen tantas ideas que su vector es un promedio difuso.
3. **Costo de tokens:** recuperar 4 chunks de 1000 chars = ~4000 chars = ~1000 tokens de contexto. Manageable y económico.

¿Por qué `chunk_overlap=200`?

El overlap del 20% previene la pérdida de información en los bordes.

Imaginá un párrafo que dice: '...la temperatura debe mantenerse entre 20° y 25°C para garantizar la estabilidad del compuesto.' Si esta oración queda partida entre dos chunks (la primera mitad al final del chunk 1, la segunda al inicio del chunk 2), ninguno de los dos contiene la información completa. El overlap de 200 chars garantiza que esos 200 últimos caracteres del chunk 1 también aparecen al inicio del chunk 2.

'Recursive' en el nombre — estrategia de división

El splitter intenta dividir primero por párrafos (doble salto de línea), luego por oraciones (punto), luego por palabras, y finalmente por caracteres. Esto respeta la estructura natural del texto: prefiere no cortar en medio de una oración si puede evitarlo.

OpenAIEmbeddings — De texto a vector

Llama a la API de OpenAI con el modelo **text-embedding-ada-002** (el default al momento de escritura). Por cada fragmento de texto, retorna un vector de **1536 números decimales** que representa el significado semántico del texto en un espacio de alta dimensión. Textos con significados similares tendrán vectores similares (ceranos en el espacio).

Chroma.from_documents() — Vectorizar y persistir

Este método realiza dos operaciones en una llamada:

1. Llama a OpenAIEmbeddings para generar el vector de cada chunk (una llamada a la API por chunk).
2. Guarda los pares (texto, vector, metadata) en ChromaDB usando SQLite y archivos de segmento binarios en el directorio CHROMA_PERSIST_DIR.

Costo de la ingesta: Ingestar un PDF de 168 chunks implica 168 llamadas al endpoint de embeddings de OpenAI. Con text-embedding-ada-002, el costo es aproximadamente \$0.0001 por 1000 tokens, lo que hace que ingestar documentos promedio cueste fracciones de centavo.

3.3 app/query.py — Consulta RAG

Este módulo implementa el pipeline de consulta: recibe una pregunta, la busca en ChromaDB, recupera contexto relevante, construye un prompt y genera la respuesta. Es donde ocurre la 'magia' del RAG.

```
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_community.vectorstores import Chroma
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from app.config import OPENAI_API_KEY, CHROMA_PERSIST_DIR, OPENAI_MODEL

PROMPT_TEMPLATE = """
Eres un asistente experto en análisis de documentos.
Usa únicamente el siguiente contexto extraído del documento para responder la pregunta.
Si la respuesta no se encuentra en el contexto, indícalo claramente en español.

Contexto:
{context}

Pregunta:
{question}

Responde de forma clara y concisa en español.
"""

def query_documents(question: str, collection_name: str) -> dict:
    try:
        embeddings = OpenAIEmbeddings(openai_api_key=OPENAI_API_KEY)

        vectorstore = Chroma(
            collection_name=collection_name,
            persist_directory=CHROMA_PERSIST_DIR,
            embedding_function=embeddings,
        )

        results = vectorstore.similarity_search(question, k=4)

        if not results:
            return {"answer": "No se encontraron resultados...", "sources": []}

        context = "\n\n".join([doc.page_content for doc in results])

        prompt = ChatPromptTemplate.from_template(PROMPT_TEMPLATE)
        llm = ChatOpenAI(openai_api_key=OPENAI_API_KEY, model=OPENAI_MODEL)
        chain = prompt | llm | StrOutputParser()

        answer = chain.invoke({"context": context, "question": question})

        sources = [{"page_content": doc.page_content, "metadata": doc.metadata}
                    for doc in results]

        return {"answer": answer, "sources": sources}

    except Exception as e:
        return {"answer": f"Error: {str(e)}", "sources": []}
```

similarity_search(question, k=4) — La búsqueda vectorial

Este método convierte la pregunta en un vector usando el mismo modelo de embeddings y busca los k fragmentos más cercanos en el espacio vectorial. La similitud se mide con cosine similarity (ver Sección 5.2).

¿Por qué k=4?

$k=4 \text{ chunks} \times \sim 1000 \text{ chars/chunk} = \sim 4000 \text{ chars} \approx 1000 \text{ tokens}$ de contexto. Es el balance óptimo entre:

- **Suficiente contexto:** con $k=1$ podría faltar información si la respuesta está distribuida en varios párrafos.
- **Sin ruido excesivo:** con $k=10$, el contexto incluiría fragmentos poco relevantes que confunden al LLM y elevan el costo.
- **Costo de tokens:** 1000 tokens de contexto a GPT-3.5 cuesta $\sim \$0.001$ por consulta.

EL PROMPT_TEMPLATE — Ingeniería de prompts

El prompt tiene tres instrucciones críticas:

1. **'Eres un asistente experto en análisis de documentos'** — establece el rol del modelo.
2. **'Usa únicamente el siguiente contexto'** — evita que el modelo use su conocimiento previo y se invente respuestas no fundamentadas en el documento.
3. **'Si la respuesta no se encuentra en el contexto, indícalo'** — fuerza honestidad ante preguntas que el documento no responde.

LCEL: El operador pipe (|)

```
chain = prompt | llm | StrOutputParser()
```

LCEL (LangChain Expression Language) usa el operador `|` (pipe) de Python para encadenar componentes. Cada componente recibe la salida del anterior como entrada. El flujo es:

1. **prompt:** recibe `{context, question}` → retorna un `ChatPromptValue`
2. **llm:** recibe el prompt formateado → retorna un `AIMessage` (objeto de LangChain)
3. **StrOutputParser:** recibe el `AIMessage` → extrae el texto como string Python

¿Por qué LCEL en lugar de RetrievalQA?

RetrievalQA es la versión antigua de LangChain para hacer RAG. Funciona pero es una 'caja negra': oculta lo que pasa internamente. LCEL es explícito — cada paso es visible en el código. Esto hace el sistema más fácil de depurar, modificar y entender. Es el approach moderno recomendado por LangChain desde v0.2.

3.4 app/main.py — API FastAPI

main.py es el punto de entrada del backend. Define la aplicación FastAPI, configura el middleware de CORS, y expone los cuatro endpoints que el frontend utiliza para interactuar con el sistema.

Configuración inicial de FastAPI:

```
app = FastAPI(
    title="RAG Document Assistant",
    description="API para ingestar documentos PDF y consultarlos mediante RAG.",
    version="0.1.0",
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],      # Acepta peticiones de cualquier origen (localhost:3000, etc.)
    allow_credentials=True,
    allow_methods=["*"],     # GET, POST, PUT, DELETE, OPTIONS, etc.
    allow_headers=["*"],     # Authorization, Content-Type, etc.
)

os.makedirs(UPLOAD_DIR, exist_ok=True)      # Crea /uploads si no existe
os.makedirs(CHROMA_PERSIST_DIR, exist_ok=True) # Crea /chroma_db si no existe
```

¿Por qué FastAPI y no Flask o Django?

- **Async nativo:** FastAPI está construido sobre ASGI (Asynchronous Server Gateway Interface). Las operaciones de I/O (leer archivos, llamar a APIs externas) no bloquean el servidor.
- **Validación automática con Pydantic:** al declarar QueryRequest(BaseModel), FastAPI valida automáticamente que el JSON entrante tiene los campos correctos y los tipos correctos.
- **Documentación automática:** FastAPI genera /docs (Swagger UI) y /redoc sin código adicional.
- **Type hints nativos:** usa las anotaciones de tipo de Python para todo lo anterior.

¿Qué es CORS y por qué habilitarlo?

CORS (Cross-Origin Resource Sharing) es un mecanismo de seguridad de los navegadores que bloquea peticiones HTTP desde un origen (ej: http://localhost:3000) hacia otro origen (ej: http://localhost:8000) a menos que el servidor lo permita explícitamente.

Sin CORS habilitado, cuando el frontend Next.js intente hacer fetch() a la API de FastAPI, el navegador bloqueará la petición. Con allow_origins=["*"] lo permitimos desde cualquier origen. En producción, se debería restringir al dominio real del frontend.

Los cuatro endpoints:

GET /health

```
@app.get("/health")
async def health_check():
    return {"status": "ok"}
```

El endpoint más simple. Su único propósito es verificar que el servidor está corriendo. Es útil para sistemas de monitoreo, load balancers y para depuración rápida.

POST /ingest

```
@app.post("/ingest")
async def ingest_document(file: UploadFile = File(...)):
    if not file.filename.endswith(".pdf"):
        raise HTTPException(status_code=400, detail="Solo se aceptan archivos PDF.")
    file_path = os.path.join(UPLOAD_DIR, file.filename)
    with open(file_path, "wb") as buffer:
        shutil.copyfileobj(file.file, buffer) # Streaming: no carga todo en memoria
    result = ingest_pdf(file_path)
    return result
```

UploadFile es el tipo de FastAPI para archivos subidos via multipart/form-data. **shutil.copyfileobj()** copia el stream del archivo directamente al disco en bloques (streaming), lo que permite manejar PDFs grandes sin agotar la memoria RAM.

POST /query

```
class QueryRequest(BaseModel):
    question: str
    collection_name: str

@app.post("/query")
async def query_document(request: QueryRequest):
    if not request.question.strip():
        raise HTTPException(status_code=400, detail="La pregunta no puede estar vacía.")
    return query_documents(request.question, request.collection_name)
```

BaseModel de Pydantic actúa como esquema: si el cliente envía JSON sin 'question' o con un tipo incorrecto, FastAPI retorna automáticamente un error 422 descriptivo antes de llegar al código de la función.

GET /collections

```
@app.get("/collections")
async def list_collections():
    client = chromadb.PersistentClient(path=CHROMA_PERSIST_DIR)
    collections = client.list_collections()
    names = [col.name for col in collections]
    return {"collections": names}
```


Usa el cliente nativo de ChromaDB para listar las colecciones existentes. Más confiable que inspeccionar el sistema de archivos, porque ChromaDB no guarda una carpeta por colección con el nombre legible — usa IDs internos.

4. Flujo Completo de una Request

4.1 Fase de Ingesta: de PDF a Vector

Paso 1: El usuario sube el PDF

El frontend hace POST /ingest con el archivo en multipart/form-data. FastAPI recibe el stream y lo delega al endpoint ingest_document().

Paso 2: Validación

FastAPI verifica que el archivo termina en .pdf. Si no, retorna HTTP 400.

Paso 3: Guardado en disco

shutil.copyfileobj() escribe el PDF en /uploads/nombre.pdf de forma streaming.

Paso 4: Carga de páginas

PyPDFLoader lee el PDF y retorna una lista de Documents, uno por página, cada uno con su metadata.

Paso 5: Chunking

RecursiveCharacterTextSplitter divide las páginas en fragmentos de máximo 1000 chars con 200 de overlap. Un PDF de 67 páginas genera ~168 chunks.

Paso 6: Embedding

Para cada chunk, OpenAIEmbeddings llama a la API de OpenAI y obtiene un vector de 1536 dimensiones. Son 168 llamadas API en secuencia.

Paso 7: Persistencia

Chroma.from_documents() guarda los 168 pares (chunk, vector) en ChromaDB bajo la colección 'nombre-del-pdf'.

Paso 8: Respuesta

FastAPI retorna {filename, total_pages, total_chunks, collection_name} al frontend.

4.2 Fase de Consulta: de Pregunta a Respuesta

Paso 1: El usuario escribe una pregunta

El frontend hace POST /query con {question: '¿De qué trata?', collection_name: 'Resumen-Libro'}.

Paso 2: Validación Pydantic

FastAPI deserializa el JSON y valida el esquema QueryRequest automáticamente.

Paso 3: Vectorización de la pregunta

OpenAIEmbeddings convierte la pregunta en un vector de 1536 dimensiones. Una sola llamada API.

Paso 4: Similarity search

ChromaDB calcula la cosine similarity entre el vector de la pregunta y los 168 vectores almacenados. Retorna los 4 más cercanos (k=4).

Paso 5: Construcción del contexto

Los `page_content` de los 4 chunks se unen con doble salto de línea para formar el bloque de contexto.

Paso 6: Formateo del prompt

`ChatPromptTemplate` rellena `{context}` y `{question}` en la plantilla `PROMPT_TEMPLATE`.

Paso 7: Generación con GPT

`ChatOpenAI` envía el prompt a GPT-3.5-turbo (o el modelo configurado). GPT lee el contexto y genera la respuesta.

Paso 8: Parsing

`StrOutputParser` extrae el texto plano del `AIMessage` que retorna GPT.

Paso 9: Construcción de fuentes

Se construye la lista de `sources` con `page_content` y `metadata` de cada uno de los 4 chunks recuperados.

Paso 10: Respuesta

FastAPI retorna `{answer: '...', sources: [{page_content, metadata}, ...]}` al frontend.

5. Conceptos Clave en Profundidad

5.1 Embeddings y Representación Vectorial

Un embedding es una representación numérica de un texto en forma de vector. Un vector es simplemente una lista de números decimales.

```
# Un embedding de la frase "el perro corre" podría verse así (simplificado a 8 dims):  
embedding = [0.234, -0.891, 0.445, 0.012, -0.334, 0.789, -0.123, 0.567]
```

```
# En realidad, text-embedding-ada-002 genera vectores de 1536 dimensiones:  
embedding_real = [0.002, -0.045, 0.123, ..., 0.089] # 1536 números
```

Analogía de las coordenadas: Así como GPS usa 3 números (latitud, longitud, altitud) para ubicar cualquier punto en el planeta, un embedding usa 1536 números para ubicar cualquier texto en un 'planeta del significado'. Textos con significados similares tienen coordenadas cercanas. 'El perro corre' y 'El can galopa' están cerca. 'El perro corre' y 'La economía global' están lejos.

¿Cómo aprende el modelo a asignar coordenadas?

El modelo de embeddings fue entrenado con enormes cantidades de texto. Aprendió que palabras que aparecen en los mismos contextos tienen significados similares. 'Rey' y 'Reina' aparecen en contextos parecidos, así que sus vectores son similares. Esta propiedad permite operaciones algebraicas famosas: $\text{vector}(\text{Rey}) - \text{vector}(\text{Hombre}) + \text{vector}(\text{Mujer}) \approx \text{vector}(\text{Reina})$.

5.2 Similarity Search y Cosine Similarity

Cuando buscamos los chunks más relevantes para una pregunta, calculamos qué tan similares son sus vectores. La métrica más usada es la **cosine similarity**.

```
# Cosine similarity mide el ángulo entre dos vectores (no su distancia)  
# Resultado entre -1 y 1:  
# 1.0 = vectores idénticos (misma dirección)  
# 0.0 = vectores perpendiculares (sin relación semántica)  
# -1.0 = vectores opuestos (significados opuestos)
```

```
cosine_similarity(vector_pregunta, vector_chunk_1) = 0.87 # Muy relevante  
cosine_similarity(vector_pregunta, vector_chunk_2) = 0.23 # Poco relevante  
cosine_similarity(vector_pregunta, vector_chunk_3) = 0.79 # Bastante relevante  
cosine_similarity(vector_pregunta, vector_chunk_4) = 0.81 # Relevante
```


- **Contexto dinámico:** los 4 chunks recuperados, insertados como {context}.
- **Pregunta:** la consulta del usuario, insertada como {question}.

5.5 LCEL: LangChain Expression Language

LCEL es la forma moderna de componer pipelines en LangChain. Usa el operador `|` (pipe) de Python para encadenar componentes:

```
# Pipeline RAG en LCEL:
chain = prompt | llm | StrOutputParser()

# Equivale a:
prompt_value = prompt.invoke({"context": context, "question": question})
ai_message   = llm.invoke(prompt_value)
answer_string = StrOutputParser().invoke(ai_message)

# LCEL también soporta streaming, async, y batch con la misma sintaxis:
async for chunk in chain.astream({"context": ctx, "question": q}):
    print(chunk, end="", flush=True)
```

Ventajas de LCEL sobre RetrievalQA (el approach antiguo):

- Explícito: cada paso es visible en el código, sin magia oculta.
- Composable: podés insertar pasos intermedios (logging, transformación) con un `|` adicional.
- Consistente: la misma API para invoke, stream, batch y async.
- Debuggeable: si algo falla, sabés exactamente en qué paso.

6. Dependencias y Librerías

Cada dependencia del archivo requirements.txt cumple un rol específico e irremplazable en el sistema:

fastapi

Framework web async para Python. Construye la API REST con validación automática, documentación Swagger y soporte para WebSockets. Elegido sobre Flask por su tipado nativo y rendimiento.

uvicorn[standard]

Servidor ASGI que ejecuta la aplicación FastAPI. El extra [standard] incluye watchfiles (recarga automática en dev) y httptools (parser HTTP más rápido). Sin uvicorn, FastAPI no puede recibir peticiones HTTP.

langchain

Framework de orquestación para aplicaciones de IA. Provee abstracciones para LLMs, embeddings, vectorstores y chains. Actúa como 'pegamento' entre los componentes.

langchain-openai

Paquete de LangChain específico para OpenAI. Provee OpenAIEmbeddings (genera vectores) y ChatOpenAI (llama al chat model). Separado del core para reducir dependencias.

langchain-community

Contiene integraciones de la comunidad: PyPDFLoader (carga PDFs) y Chroma (vectorstore). Separado porque estas integraciones tienen sus propias dependencias.

langchain-text-splitters

Paquete independiente con los splitters de texto. Incluye RecursiveCharacterTextSplitter. Fue separado de langchain core en v0.2 para reducir el tamaño del paquete base.

chromadb

Base de datos vectorial embebida y persistente. Funciona sin servidor externo (todo local), lo que la hace ideal para proyectos de aprendizaje y prototipos. Alternativas: Pinecone (cloud), Weaviate, Qdrant.

pypdf

Librería Python para leer archivos PDF. PyPDFLoader de LangChain la usa internamente. Extrae texto página por página y metadata del documento.

python-dotenv

Lee archivos .env y carga los pares CLAVE=valor en el entorno del proceso. Permite separar la configuración del código sin hardcodear secretos.

openai

SDK oficial de OpenAI para Python. LangChain lo usa internamente para llamar a los endpoints de embeddings y chat completion. Sin este paquete, langchain-openai no funciona.

python-multipart

Necesario para que FastAPI procese uploads de archivos en formato multipart/form-data. Sin este paquete, el endpoint POST /ingest falla al intentar recibir el PDF.

7. Decisiones de Diseño

Cada decisión técnica del proyecto tiene una justificación. Esta sección documenta las principales alternativas consideradas y por qué se eligió cada approach.

ChromaDB vs. Pinecone / Weaviate / Qdrant

Se eligió ChromaDB por ser completamente local (sin servidor externo, sin API key adicional, sin costo), persistente en disco mediante SQLite, y sencilla de configurar. Para un proyecto de aprendizaje, eliminar la dependencia de un servicio cloud externo reduce la fricción. Pinecone sería la elección correcta en producción por su escalabilidad.

OpenAI para embeddings y LLM vs. modelos locales

OpenAI ofrece la mejor relación calidad/facilidad de uso. text-embedding-ada-002 produce embeddings de alta calidad con una sola llamada API. GPT-3.5-turbo es económico y capaz. La alternativa sería usar modelos locales (Llama3, mistral via Ollama) que eliminarían el costo de API pero requerirían hardware potente y setup complejo.

FastAPI vs. Flask vs. Django

FastAPI fue elegido por async nativo (crítico para no bloquear el servidor durante llamadas a OpenAI que pueden tardar varios segundos), validación automática con Pydantic, y documentación Swagger incluida. Flask requeriría plugins adicionales para async. Django sería excesivo para una API simple.

LCEL vs. RetrievalQA de LangChain

LCEL (LangChain Expression Language) es el approach moderno y explícito. RetrievalQA es legacy: oculta los pasos intermedios y hace difícil la depuración. Con LCEL, cada componente del pipeline es visible: prompt | llm | parser. Más código, pero mucho más comprensible.

chunk_size=1000 vs. otros tamaños

1000 chars es el balance entre contexto (suficiente para una idea completa) y precisión del retrieval (un chunk muy largo tiene un embedding 'difuso' que mezcla muchos temas). Para documentos técnicos densos podría reducirse a 500. Para narrativa podría subirse a 1500.

Nombre del PDF como collection_name

Usar el nombre del archivo (sin extensión) como nombre de colección en ChromaDB tiene una ventaja práctica: es predecible y legible. El usuario sabe que subió 'manual.pdf' y la colección se llama 'manual'. La alternativa (UUID) sería más robusta pero requeriría una capa de mapeo nombre→uuid.

shutil.copyfileobj vs. file.read()

`shutil.copyfileobj()` copia el stream del archivo en bloques (chunks de 16KB por defecto). `file.read()` cargaría todo el PDF en memoria RAM antes de guardarlo. Para PDFs grandes (100MB+), `file.read()` puede agotar la memoria. `copyfileobj` es la práctica correcta.

8. Glosario de Términos Técnicos

API (Application Programming Interface)

Interfaz que permite que dos sistemas de software se comuniquen. En este proyecto, FastAPI expone una API REST que el frontend usa para subir PDFs y hacer consultas.

Async / Await

Modelo de programación que permite ejecutar operaciones de I/O (llamadas HTTP, lectura de archivos) sin bloquear el hilo principal del servidor. FastAPI lo usa nativamente.

Chunk

Fragmento de texto resultante de dividir un documento largo. En este proyecto, chunks de máximo 1000 caracteres con 200 de solapamiento con el chunk anterior.

ChromaDB

Base de datos vectorial embebida. Almacena pares (texto, vector) y permite búsqueda por similitud semántica. Persiste en disco usando SQLite y archivos binarios.

CORS (Cross-Origin Resource Sharing)

Mecanismo de seguridad de navegadores que controla qué orígenes pueden hacer peticiones HTTP a un servidor. Debe habilitarse para que el frontend pueda llamar al backend.

Cosine Similarity

Métrica que mide el ángulo entre dos vectores. Resultado entre -1 y 1. En RAG se usa para encontrar chunks cuyo embedding es más similar al embedding de la pregunta.

Embedding

Representación numérica (vector) de un texto. Textos semánticamente similares tienen embeddings similares (vectores cercanos). `text-embedding-ada-002` genera vectores de 1536 dimensiones.

FastAPI

Framework web moderno para Python basado en ASGI. Provee validación automática con Pydantic, documentación Swagger y soporte async nativo.

Fine-tuning

Proceso de reentrenar un modelo de lenguaje con datos específicos para que aprenda nuevos conocimientos. Más costoso y complejo que RAG, pero útil para cambiar el estilo o personalidad del modelo.

LangChain

Framework de orquestación para aplicaciones de IA. Conecta LLMs, embeddings, vectorstores y otras herramientas mediante abstracciones reutilizables.

LCEL (LangChain Expression Language)

Sintaxis moderna de LangChain para construir pipelines usando el operador `|`. Permite encadenar componentes de forma explícita y composable.

LLM (Large Language Model)

Modelo de lenguaje de gran escala. En este proyecto, GPT-3.5-turbo o GPT-4 de OpenAI. Genera texto coherente y contextualizado en lenguaje natural.

Multipart/form-data

Formato de encoding HTTP usado para subir archivos. El frontend envía el PDF en este formato; FastAPI lo recibe como `UploadFile`.

OpenAI Embeddings

Servicio de OpenAI (modelo `text-embedding-ada-002`) que convierte texto en vectores de 1536 dimensiones. Costo: ~\$0.0001 por 1000 tokens.

Overlap

Solapamiento entre chunks consecutivos. Garantiza que ideas que cruzan el borde entre dos chunks no se pierdan en ninguno.

Pydantic

Librería Python de validación de datos mediante type hints. FastAPI la usa para validar automáticamente los cuerpos JSON de las requests.

PyPDFLoader

Cargador de PDFs de LangChain que usa la librería `pypdf` internamente. Retorna una lista de `Documents` con el texto y metadata de cada página.

RAG (Retrieval-Augmented Generation)

Arquitectura de IA que combina búsqueda semántica (retrieval) con generación de texto (generation). Permite a un LLM responder preguntas sobre documentos privados.

RecursiveCharacterTextSplitter

Splitter de texto de LangChain que divide por párrafos, luego oraciones, luego palabras, respetando la estructura natural del texto.

Similarity Search

Búsqueda de los vectores más cercanos a uno dado. ChromaDB la implementa calculando cosine similarity entre el vector de la pregunta y todos los vectors de la colección.

Vector / Vector Space

Un vector es una lista de números. Un espacio vectorial es el 'universo' donde viven todos esos vectores. En RAG, cada texto tiene un vector que lo ubica en ese espacio.

Vectorstore

Base de datos especializada en almacenar y buscar vectores. ChromaDB es el vectorstore de este proyecto.

uvicorn

Servidor ASGI para Python. Ejecuta la aplicación FastAPI y maneja las conexiones HTTP entrantes. Se arranca con 'uvicorn app.main:app --reload'.